UNIFIL · CENTRO UNIVERSITÁRIO FILADÉLFIA




# HYPE CARDS

## Sistema web de controle de estoque para loja de Pokémon TCG

Documentação técnica do projeto: requisitos,
modelagem UML e banco de dados

**Murillo Daigder**

Londrina – PR · 2026

# Sumário


1 Introdução

1.1 O sistema e o problema que resolve

1.2 Objetivos

1.3 Stack utilizada

1.4 Arquitetura

2 Desenvolvimento

2.1 Requisitos funcionais e não funcionais

2.2 Diagrama de casos de uso

2.3 Diagrama Entidade-Relacionamento (DER)

2.4 Diagrama de atividade

2.5 Diagrama de classes

2.6 Diagramas de sequência

3 Conclusão

4 Referências

A Apêndice A · Fluxo de autenticação

# 1. Introdução

O Hype Cards é um sistema web que controla o estoque de uma loja de produtos Pokémon TCG (Trading Card Game) e registra cada entrada, saída e venda com data, valor e responsável. Ele substitui planilhas e anotações soltas por um cadastro único, acessado pelo navegador.

## 1.1 O sistema e o problema que resolve

Uma loja de cartas colecionáveis vende três tipos de mercadoria com características bem diferentes:

- **Cartas avulsas**: cada uma tem coleção, número, raridade, condição de conservação e idioma, e uma única carta pode valer milhares de reais.
- **Produtos selados**: boosters, Elite Trainer Boxes, displays e latas, sempre ligados a uma coleção.
- **Acessórios**: sleeves, deck boxes, pastas e playmats, organizados por categoria e marca.

Sem um sistema, o lojista não sabe com segurança quanto tem em estoque, quanto vendeu no mês, qual foi o lucro nem quem fez cada movimentação. O Hype Cards resolve isso com cadastro por categoria, alerta de estoque baixo, histórico completo de movimentações e um painel com faturamento, lucro e itens mais vendidos.

O público é a própria loja: o **administrador** (dono ou gerente), que cadastra itens e usuários, e o **vendedor**, que consulta o estoque e registra vendas e entradas.

## 1.2 Objetivos

- Centralizar o cadastro de coleções, cartas, produtos selados e acessórios, com foto de cada item.
- Registrar toda entrada e saída de estoque com motivo, valor e usuário, formando um histórico auditável.
- Calcular automaticamente o valor e o custo do estoque, o faturamento e o lucro do mês.
- Avisar quando um item fica abaixo do estoque mínimo.
- Proteger o acesso com login e dois perfis de permissão.
- Permitir exportar dados para planilha (CSV) e gerar backup completo.

## 1.3 Stack utilizada

**Figura 1 – Tecnologias utilizadas no Hype Cards**




Fonte: elaborado pelo autor (2026), com logotipos do Simple Icons.

**Quadro 1 – Camadas e tecnologias**

| CAMADA | TECNOLOGIA | USO NO PROJETO |
|---|---|---|
| Servidor | Node.js 22 ou superior | Executa a aplicação; módulos nativos `node:sqlite`, `node:crypto`, `node:zlib` e `node:test` |
| API | Express 5 | Rotas REST, arquivos estáticos e tratamento de erros assíncronos |
| Banco principal | PostgreSQL 18 (driver `pg`) | Banco relacional usado em produção, com transações e trava de linha |
| Banco alternativo | SQLite (`node:sqlite`) | Banco em arquivo único, usado quando não há PostgreSQL configurado |
| Interface | HTML5, CSS3 e JavaScript puro | Telas, tema claro/escuro e chamadas à API com `fetch` |
| Testes | node:test | 18 testes automatizados da API, executados nos dois bancos |

O projeto tem apenas duas dependências externas (`express` e `pg`). Senhas usam hash `scrypt`, o backup em ZIP é gerado com `zlib`, e a exportação CSV é feita sem bibliotecas adicionais.

## 1.4 Arquitetura

O sistema segue a arquitetura cliente-servidor em três camadas: o navegador exibe as telas, a API Express aplica as regras de negócio e a camada de acesso a dados conversa com o banco. Essa última camada oferece a mesma interface para PostgreSQL e SQLite, e o banco é escolhido pela variável `DATABASE_URL` do arquivo `.env`. As fotos dos itens ficam em disco, na pasta `data/imagens`, e o banco guarda apenas o nome de cada arquivo.

# 2. Desenvolvimento

Esta seção descreve o que o sistema faz (requisitos) e como ele foi modelado: casos de uso, banco de dados, fluxo de atividades, classes e a troca de mensagens entre as partes em cada caso de uso.

## 2.1 Requisitos funcionais e não funcionais

O sistema tem 18 requisitos funcionais e 12 não funcionais. Na coluna Ator, "Todos" significa administrador e vendedor.

**Quadro 2 – Requisitos funcionais**

| CÓDIGO | REQUISITO | ATOR |
|---|---|---|
| RF01 | Autenticar o usuário com login e senha e encerrar a sessão (logout) | Todos |
| RF02 | Exigir a troca da senha no primeiro acesso e permitir alterá-la depois | Todos |
| RF03 | Gerenciar usuários: cadastrar, editar, desativar, redefinir senha e excluir, com perfil administrador ou vendedor | Administrador |
| RF04 | Manter coleções (nome, sigla, série, data de lançamento) | Administrador |
| RF05 | Manter cartas avulsas (coleção, número, raridade, condição, idioma, preço de venda, preço de custo, quantidade e estoque mínimo) | Administrador |
| RF06 | Manter produtos selados (coleção, tipo, preços, quantidade e estoque mínimo) | Administrador |
| RF07 | Manter acessórios (categoria, marca, preços, quantidade e estoque mínimo) | Administrador |
| RF08 | Enviar e remover a foto de cada item, reduzindo a imagem automaticamente antes do envio | Administrador |
| RF09 | Registrar entrada de estoque com motivo (Compra, Devolução de cliente, Ajuste de inventário) e custo pago | Todos |
| RF10 | Registrar saída de estoque com motivo (Venda, Avaria/Perda, Uso interno, Ajuste de inventário), guardando o preço cobrado na venda e impedindo estoque negativo | Todos |
| RF11 | Lançar automaticamente no histórico o estoque do cadastro inicial e toda alteração de quantidade feita pela edição | Sistema |

| CÓDIGO | REQUISITO | ATOR |
|---|---|---|
| RF12 | Consultar o histórico geral com filtros (item, tipo, operação, motivo e período) e o histórico de cada item | Todos |
| RF13 | Exibir o painel de resumo: vendas e lucro do mês, valor e custo do estoque, mais vendidos e itens abaixo do mínimo | Todos |
| RF14 | Buscar por nome, filtrar por coleção e por estoque baixo, ordenar por coluna e paginar as listas | Todos |
| RF15 | Alternar a listagem entre tabela e grade com fotos | Todos |
| RF16 | Exportar listas e histórico para CSV, respeitando os filtros aplicados | Todos |
| RF17 | Gerar backup em ZIP com o banco de dados e as imagens | Administrador |
| RF18 | Alternar entre tema claro e escuro, lembrando a escolha | Todos |

**Quadro 3 – Requisitos não funcionais**

| CÓDIGO | CATEGORIA | REQUISITO |
|---|---|---|
| RNF01 | Segurança | Senhas guardadas com hash `scrypt` e salt; sessão por cookie `HttpOnly` e `SameSite=Lax`, válida por 7 dias, com apenas o hash SHA-256 do token no banco |
| RNF02 | Segurança | Bloquear o login por 1 minuto após 5 tentativas erradas do mesmo usuário e IP |
| RNF03 | Segurança | Verificar o perfil em toda rota da API, não apenas esconder botões na tela |
| RNF04 | Integridade | Gravar movimentação e saldo na mesma transação, travar a linha do item durante a movimentação (PostgreSQL) e usar chaves estrangeiras e restrições `CHECK` (preço e quantidade ≥ 0) |
| RNF05 | Portabilidade | Funcionar com PostgreSQL ou SQLite sem mudança de código, escolhendo o banco pela variável `DATABASE_URL` |
| RNF06 | Desempenho | Paginar as listas (até 100 registros por página), indexar o histórico por item e por data e reduzir fotos para no máximo 1000 px em WEBP |
| RNF07 | Usabilidade | Interface em português, responsiva até telas de celular, com diálogos próprios e prévia do saldo antes de confirmar uma movimentação |
| RNF08 | Compatibilidade | Rodar em Node.js 22.13 ou superior e nos navegadores Chrome, Edge e Firefox atuais |

| CÓDIGO | CATEGORIA | REQUISITO |
|---|---|---|
| RNF09 | Manutenibilidade | Ter testes automatizados da API (18 cenários) executados nos dois bancos e no máximo duas dependências externas |
| RNF10 | Rastreabilidade | Manter o histórico legível mesmo após excluir o item ou o usuário, copiando seus nomes para a movimentação |
| RNF11 | Validação | Validar todos os dados no servidor com mensagens claras; aceitar imagens PNG, JPG, WEBP ou GIF de até 5 MB |
| RNF12 | Interoperabilidade | Gerar CSV que abre direto no Excel em português: separador `;`, vírgula decimal e codificação UTF-8 com BOM |

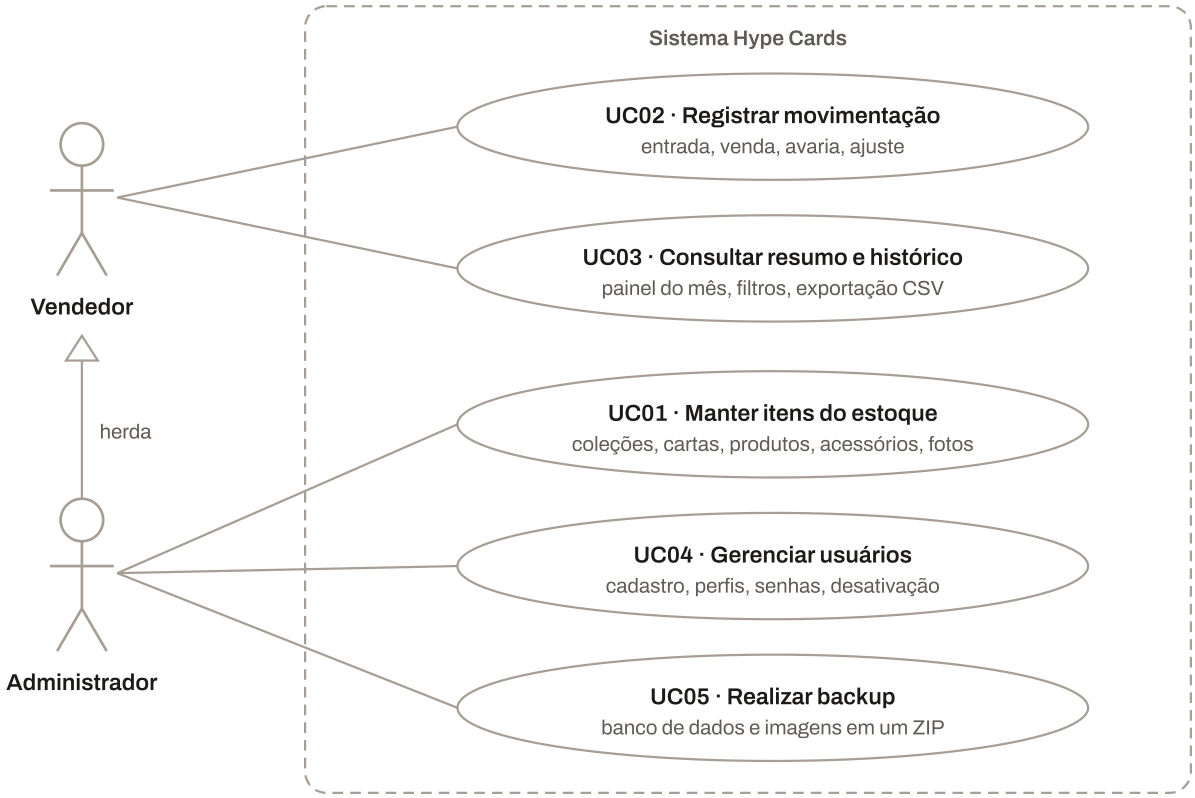
## 2.2 Diagrama de casos de uso

O sistema tem dois atores e cinco casos de uso. O administrador herda tudo o que o vendedor faz (generalização) e, além disso, mantém o cadastro de itens, gerencia usuários e realiza o backup.

**Por que o login não é um caso de uso.** O login não entrega um resultado de negócio por si só: ele é a **pré-condição** de todos os casos de uso. Por isso aparece nos requisitos (RF01 e RF02), na segurança (RNF01 e RNF02) e no Apêndice A, que mostra o fluxo de autenticação.

**Figura 2 – Diagrama de casos de uso · 2 atores, 5 casos de uso**

**O vendedor opera o estoque; o administrador também cadastra, gerencia e faz backup**




Fonte: elaborado pelo autor (2026).

**Quadro 4 – Descrição dos casos de uso**

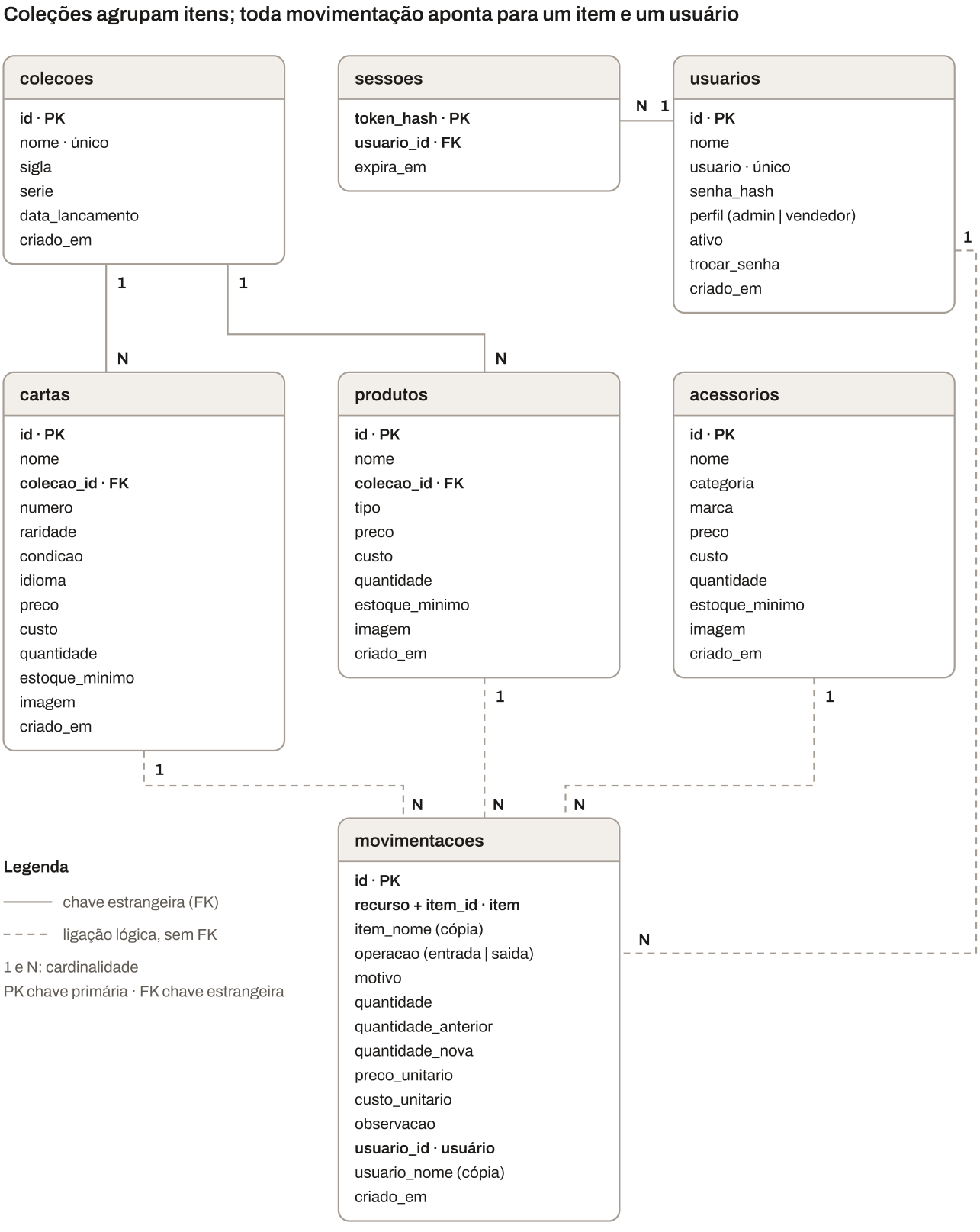
| CASO DE USO | ATORES | DESCRIÇÃO | PRÉ-CONDIÇÃO | RESULTADO |
|---|---|---|---|---|
| **UC01** · Manter itens do estoque | Administrador | Cadastra, edita e exclui coleções, cartas, produtos e acessórios, com foto opcional | Autenticado como administrador | Item salvo; quantidade inicial lançada no histórico |
| **UC02** · Registrar movimentação | Vendedor, Administrador | Registra entrada (compra, devolução, ajuste) ou saída (venda, avaria, uso interno, ajuste) com quantidade e valor | Autenticado; item cadastrado | Saldo atualizado e movimentação gravada no histórico |
| **UC03** · Consultar resumo e histórico | Vendedor, Administrador | Consulta o painel do mês, as listas com filtros e o histórico; exporta em CSV | Autenticado | Informação exibida ou arquivo CSV baixado |

| CASO DE USO | ATORES | DESCRIÇÃO | PRÉ-CONDIÇÃO | RESULTADO |
|---|---|---|---|---|
| **UC04** · Gerenciar usuários | Administrador | Cadastra, edita, desativa, redefine a senha e exclui usuários, definindo o perfil de cada um | Autenticado como administrador | Usuário salvo; no primeiro acesso, ele define a própria senha |
| **UC05** · Realizar backup | Administrador | Gera um arquivo ZIP com a cópia do banco de dados e todas as fotos dos itens | Autenticado como administrador | Arquivo de backup baixado |

## 2.3 Diagrama Entidade-Relacionamento (DER)

O banco tem sete tabelas. Cartas e produtos pertencem a uma coleção por chave estrangeira; o histórico ( `movimentacoes` ) liga-se ao item pelo par `recurso` + `item_id` e guarda uma cópia do nome do item e do usuário, para continuar legível depois de exclusões.

**Figura 3** – Diagrama Entidade-Relacionamento · 7 tabelas, 7 relacionamentos




Fonte: elaborado pelo autor (2026).

**Quadro 5 – Dicionário de dados resumido**

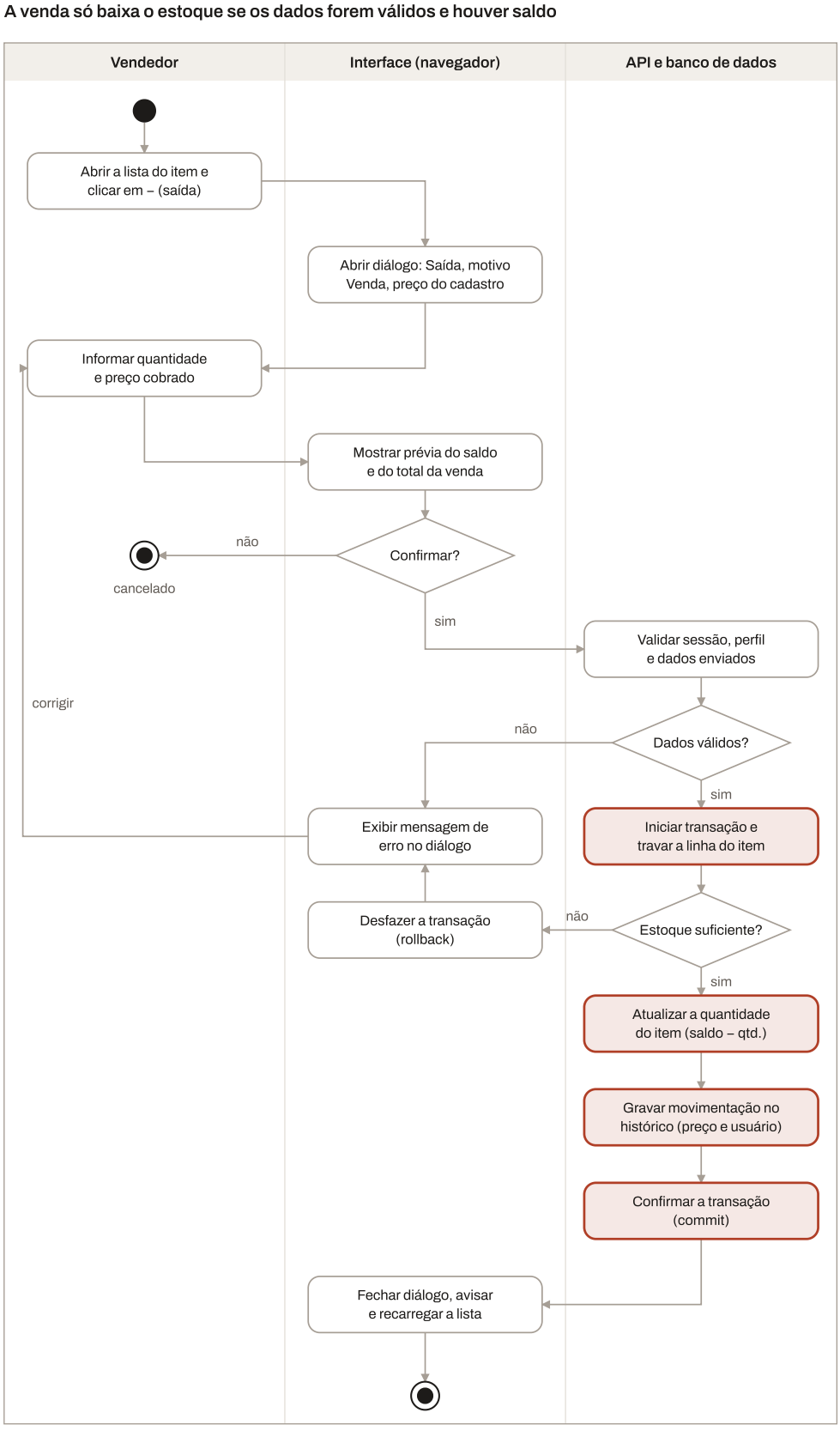
| TABELA | GUARDA | CHAVE PRIMÁRIA | RELACIONAMENTOS | REGRAS PRINCIPAIS |
|---|---|---|---|---|
| `colecoes` | Coleções de Pokémon TCG | id | 1:N com cartas e com produtos | `nome` único; não pode ser excluída com itens vinculados ( `ON DELETE RESTRICT` ) |
| `cartas` | Cartas avulsas | id | N:1 com colecoes; 1:N com movimentacoes | `preco` , `custo` , `quantidade` e `estoque_minimo` ≥ 0 ( `CHECK` ) |
| `produtos` | Produtos selados | id | N:1 com colecoes; 1:N com movimentacoes | Mesmas regras de estoque das cartas; `tipo` obrigatório |
| `acessorios` | Acessórios | id | 1:N com movimentacoes | Mesmas regras de estoque; `categoria` obrigatória |
| `movimentacoes` | Histórico de entradas e saídas | id | N:1 com o item ( `recurso` + `item_id` ) e com usuarios | `operacao` ∈ {entrada, saida}; `quantidade` > 0; índices por item e por data |
| `usuarios` | Contas de acesso | id | 1:N com sessoes e com movimentacoes | Login único sem diferenciar maiúsculas; `perfil` ∈ {admin, vendedor}; senha só como hash |
| `sessoes` | Logins ativos | token_hash | N:1 com usuarios | Excluídas junto com o usuário ( `ON DELETE CASCADE` ); expiram em 7 dias |

Os campos `criado_em` recebem a data e hora do servidor automaticamente. No PostgreSQL os ids são `IDENTITY` e os valores monetários `DOUBLE PRECISION` ; no SQLite, `AUTOINCREMENT` e `REAL` .

## 2.4 Diagrama de atividade

O diagrama mostra o fluxo mais importante do sistema, o registro de uma venda (caso de uso UC02). A baixa do estoque e a gravação do histórico acontecem dentro de uma única transação (passos destacados): se faltar saldo, nada é gravado.

**Figura 4 – Diagrama de atividade · registrar uma venda · 3 raias, 3 decisões**



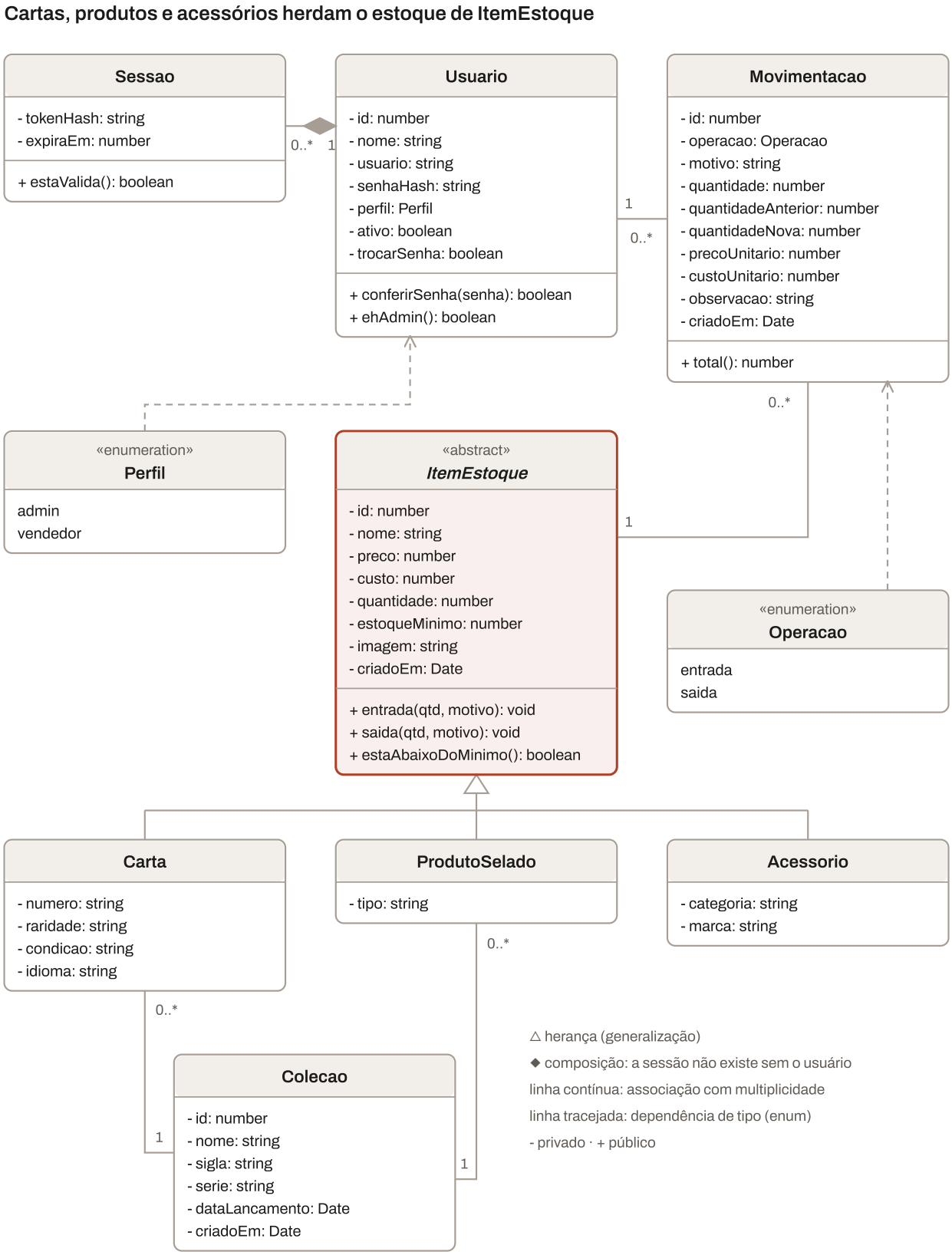

Fonte: elaborado pelo autor (2026).

Entradas de estoque (compra, devolução, ajuste) seguem o mesmo fluxo pelo botão +, sem a verificação de saldo. A trava da linha impede que duas vendas simultâneas leiam o mesmo saldo e deixem o estoque negativo.

## 2.5 Diagrama de classes

O diagrama representa o modelo conceitual das entidades. A classe abstrata `ItemEstoque` concentra o que cartas, produtos selados e acessórios têm em comum: preço, custo, quantidade, estoque mínimo, foto e as operações de entrada e saída.

**Figura 5 – Diagrama de classes · entidades do domínio · 9 classes**




Fonte: elaborado pelo autor (2026).

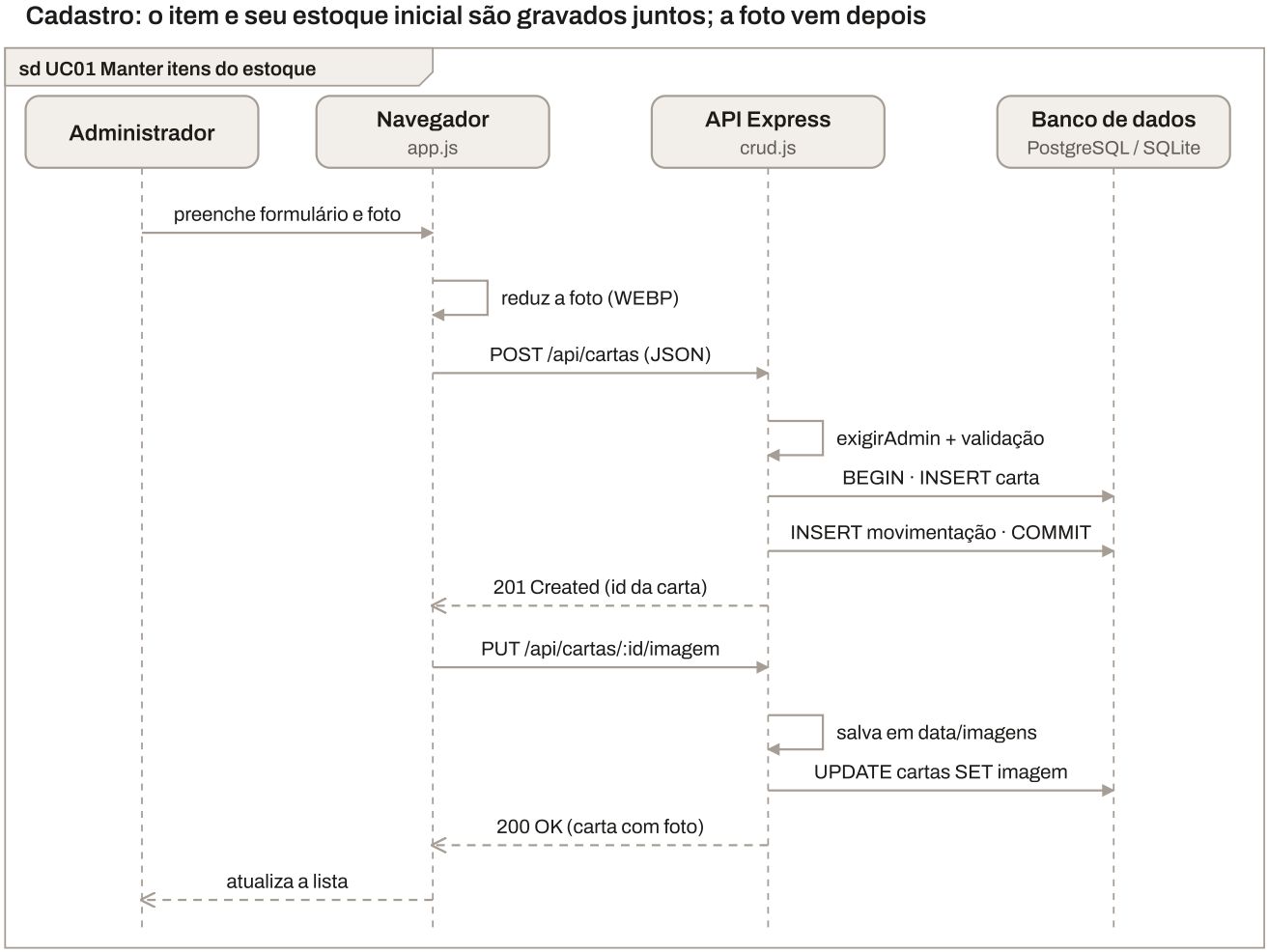
No código, escrito em JavaScript sem orientação a objetos formal, essa herança aparece como um gerador genérico de rotas ( `crud.js` ): os campos comuns são declarados uma vez ( `camposEstoque` em `recursos.js` ) e reaproveitados pelas três tabelas. Os métodos do diagrama correspondem a rotas da API, por exemplo `entrada()` e `saida()` → `PATCH /api/{recurso}/{id}/estoque`.

## 2.6 Diagramas de sequência

Cada caso de uso tem seu diagrama de sequência, com quatro participantes: o usuário, o navegador ( `app.js` ), a API Express com o módulo responsável e o banco de dados. Setas cheias são chamadas; setas tracejadas, respostas. Os quadros `alt` mostram caminhos alternativos. Todos partem de um usuário já autenticado; o fluxo de autenticação está no Apêndice A.

### UC01 · Manter itens do estoque

**Figura 6 – Sequência UC01 · cadastro de carta com foto**

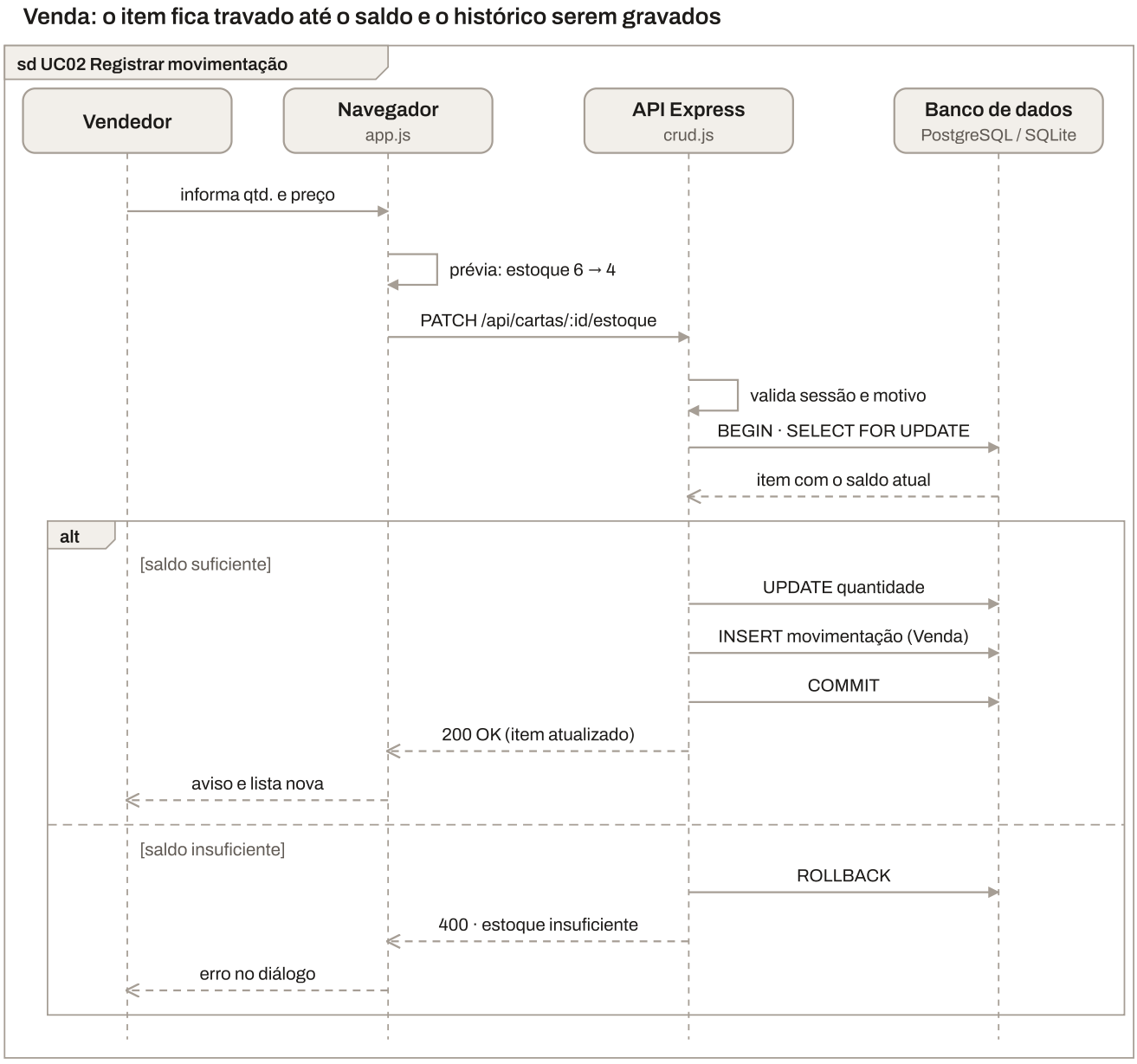



Fonte: elaborado pelo autor (2026).

O exemplo é o cadastro de uma carta. A edição ( `PUT` ) e a exclusão ( `DELETE` ) seguem o mesmo caminho; se a edição mudar a quantidade, a diferença entra no histórico como “Ajuste de inventário”.

## UC02 · Registrar movimentação

**Figura 7 – Sequência UC02 · registro de uma venda**

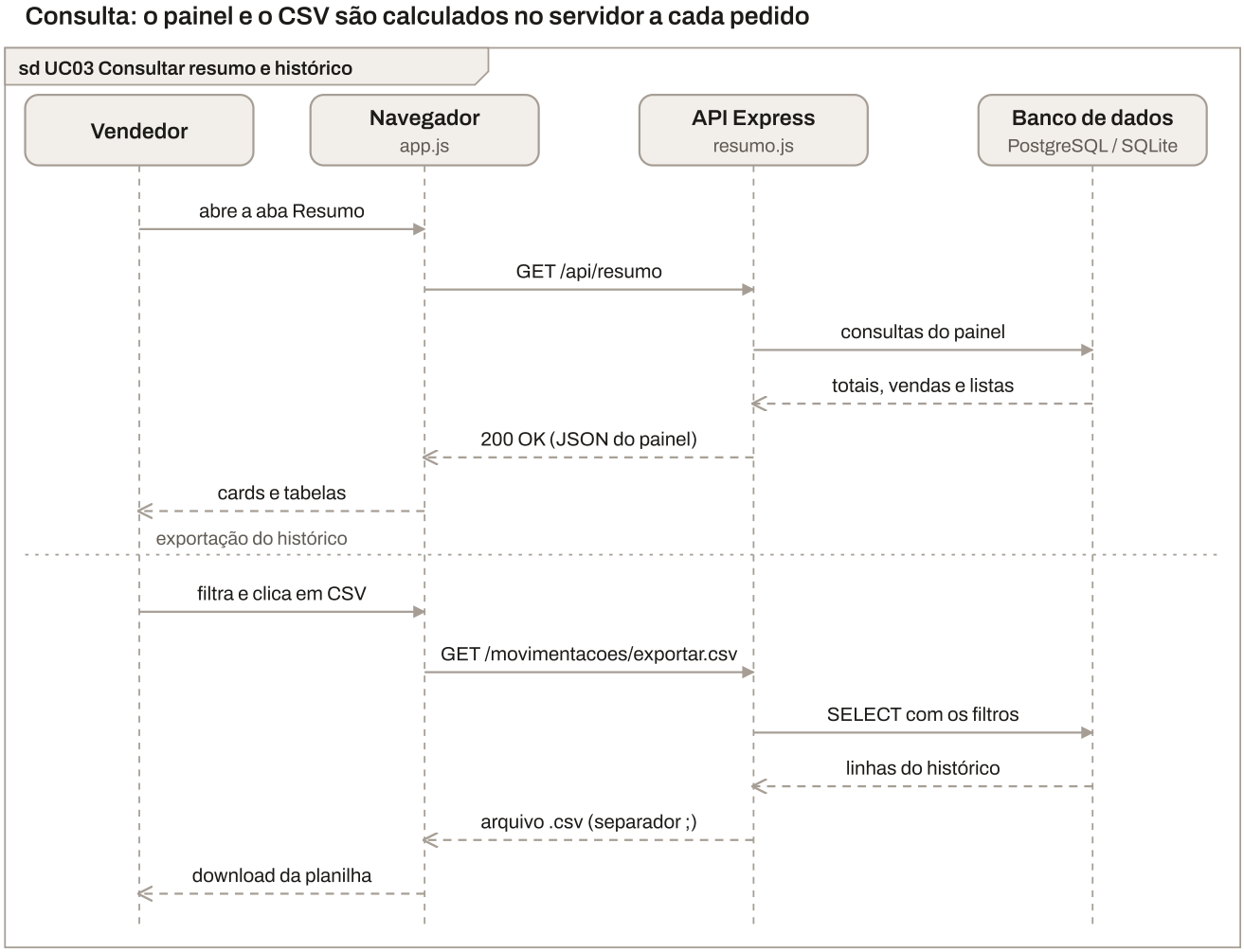



Fonte: elaborado pelo autor (2026).

No PostgreSQL, o `SELECT ... FOR UPDATE` trava a linha do item até o `COMMIT` ou o `ROLLBACK`, então duas vendas simultâneas do mesmo item são processadas uma depois da outra. No SQLite, a camada de acesso a dados enfileira as transações com o mesmo efeito.

## UC03 · Consultar resumo e histórico

**Figura 8 – Sequência UC03 · painel do mês e exportação CSV**

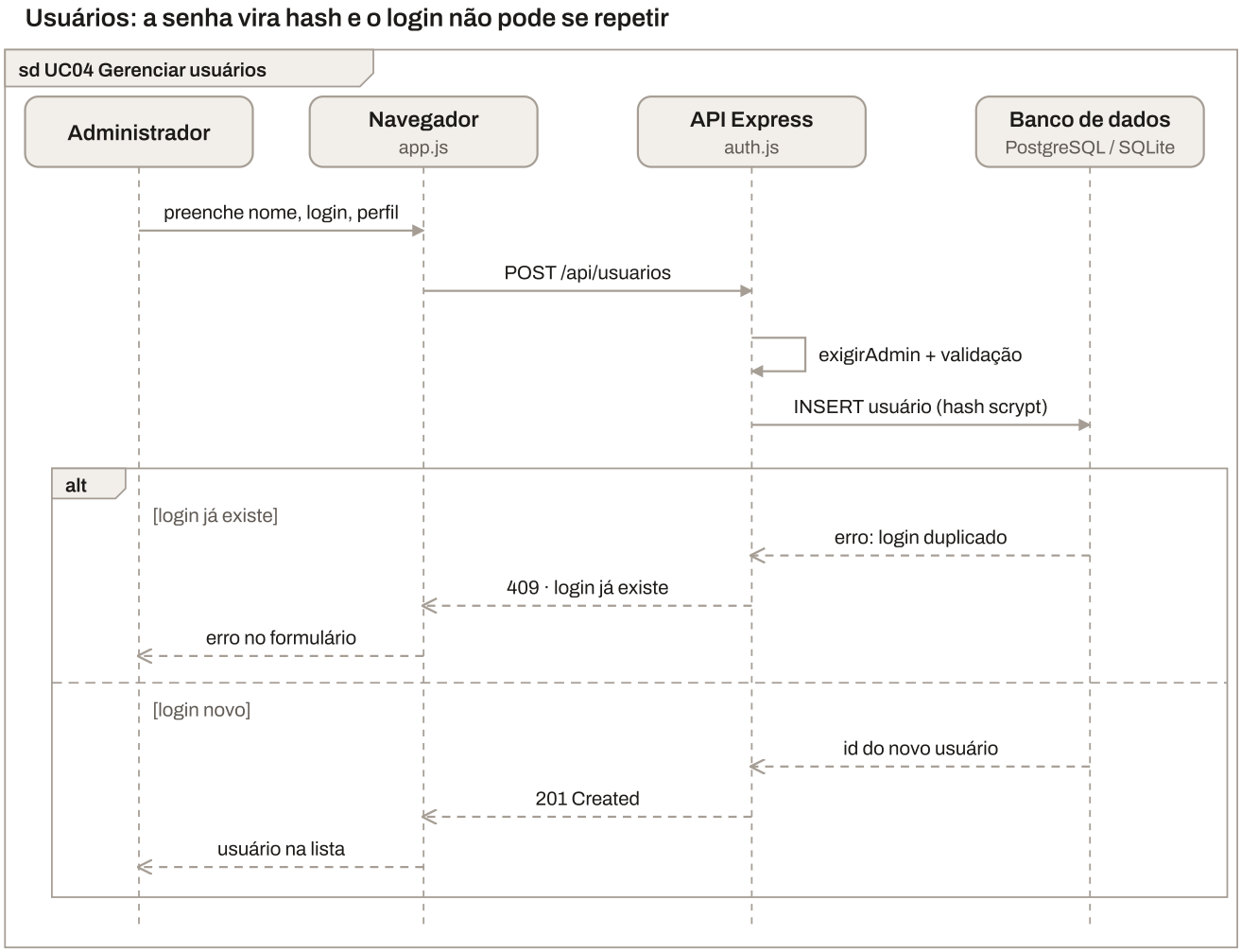



Fonte: elaborado pelo autor (2026).

O resumo executa as consultas de totais, vendas do mês, mais vendidos e estoque baixo em paralelo (`Promise.all`). O CSV usa os mesmos filtros da tela, sem paginação.

## UC04 · Gerenciar usuários

**Figura 9 – Sequência UC04 · cadastro de um usuário**

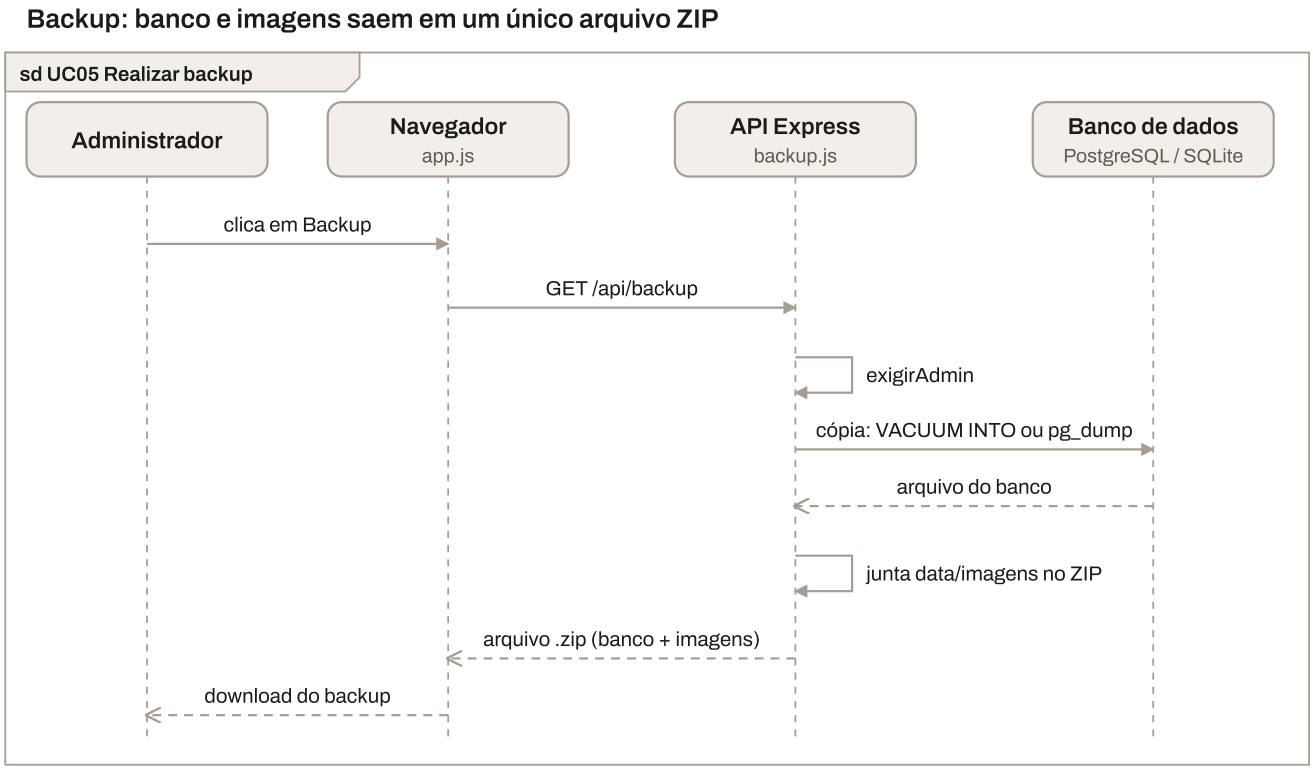



Fonte: elaborado pelo autor (2026).

O usuário criado recebe `trocar_senha = 1` e precisa definir a própria senha no primeiro acesso. A API impede excluir o próprio usuário e remover o último administrador ativo; desativar um usuário ou redefinir sua senha encerra as sessões abertas dele.

## UC05 · Realizar backup

**Figura 10 – Sequência UC05 · geração do backup**




Fonte: elaborado pelo autor (2026).

No SQLite, a cópia do banco usa `VACUUM INTO`, que gera um arquivo consistente mesmo com o sistema em uso; no PostgreSQL, o `pg_dump` gera um script SQL. Um vendedor que chame essa rota recebe a resposta 403.

# 3. Conclusão

O Hype Cards atingiu os objetivos propostos: controla o estoque das três categorias da loja, registra cada movimentação com responsável e valor, calcula faturamento e lucro do mês e roda em PostgreSQL ou SQLite. Os 18 testes automatizados passam nos dois bancos, e os dados do protótipo inicial foram migrados para o PostgreSQL sem perda.

O que aprendi com o projeto:

- **Modelagem antes do código.** Separar cartas, produtos e acessórios, mas com os mesmos campos de estoque, permitiu um único módulo genérico de cadastro. O histórico só ficou confiável depois de copiar o nome do item e do usuário para cada movimentação.
- **Transações e concorrência.** Baixar o estoque e gravar o histórico precisam acontecer juntos, ou nenhum dos dois. Com dois vendedores ao mesmo tempo, foi preciso travar a linha do item (`SELECT ... FOR UPDATE`) para o saldo não ficar negativo.
- **Segurança na prática.** Senha nunca é guardada em texto puro, o token de sessão também não, e a permissão precisa ser conferida no servidor. Esconder um botão na tela não protege nada.
- **Bancos diferentes, SQL diferente.** A migração do SQLite para o PostgreSQL mostrou diferenças reais: marcadores `?` contra `$1`, `AUTOINCREMENT` contra `IDENTITY`, `LIKE` contra `ILIKE` e códigos de erro próprios. Isolar o acesso a dados numa camada única deixou a troca restrita a poucos arquivos.
- **Testes automatizados valem o esforço.** Rodar os mesmos testes no PostgreSQL encontrou dois erros que não apareciam no SQLite: o código de erro de chave estrangeira e o caminho do `pg_dump` no Windows.
- **Experiência do usuário.** Detalhes como a prévia “Estoque: 6 → 4” antes de confirmar, a redução automática das fotos e o modo escuro fazem diferença no uso diário da loja.

Como evolução, o sistema pode ganhar emissão de recibo de venda, cadastro de clientes e fornecedores e relatórios por período.

# 4. Referências


ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

EXPRESS. **Express 5.x: API reference**. Disponível em: https://expressjs.com/en/5x/api.html. Acesso em: 14 out. 2026.

GUEDES, Gilleanes T. A. **UML 2: uma abordagem prática**. 3. ed. São Paulo: Novatec, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

MDN WEB DOCS. **HTML, CSS e JavaScript**. Mozilla. Disponível em: https://developer.mozilla.org/pt-BR/. Acesso em: 14 out. 2026.

NODE-POSTGRES. **node-postgres documentation**. Disponível em: https://node-postgres.com/. Acesso em: 14 out. 2026.

NODE.JS. **Node.js v22 documentation**. OpenJS Foundation. Disponível em: https://nodejs.org/docs/latest-v22.x/api/. Acesso em: 14 out. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (UML), version 2.5.1**. 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/. Acesso em: 14 out. 2026.

OWASP. **Password storage cheat sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 14 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 documentation**. Disponível em: https://www.postgresql.org/docs/18/. Acesso em: 14 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SIMPLE ICONS. **Simple Icons: free SVG icons for popular brands**. Disponível em: https://simpleicons.org/. Acesso em: 14 out. 2026.

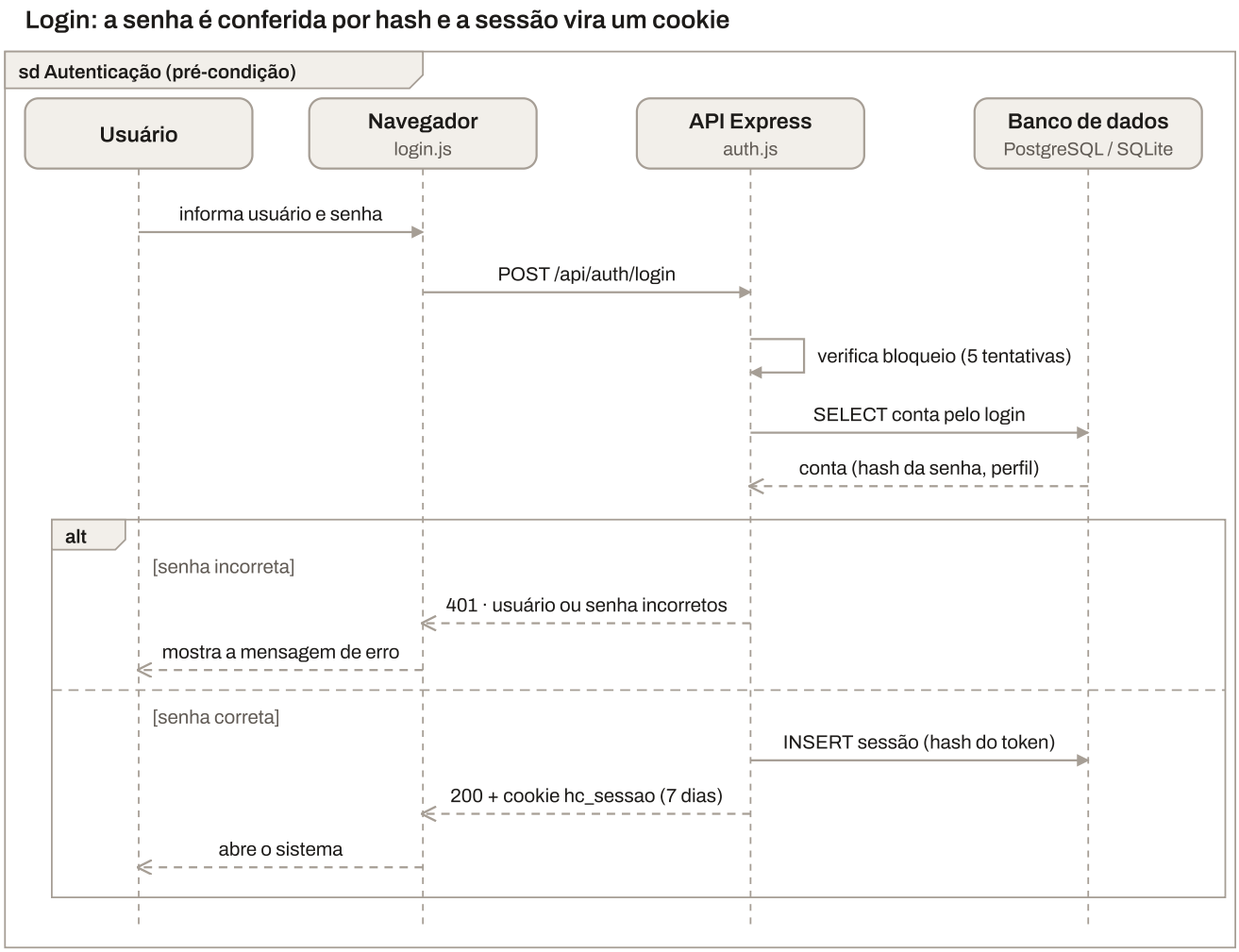
SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

SQLITE. **SQLite documentation**. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 14 out. 2026.

# Apêndice A · Fluxo de autenticação

A autenticação é a pré-condição de todos os casos de uso. O diagrama mostra o login: o servidor confere a senha pelo hash `scrypt`, bloqueia o acesso por 1 minuto após 5 tentativas erradas e, quando a senha está correta, abre uma sessão de 7 dias guardada em cookie. No banco fica apenas o hash SHA-256 do token da sessão.

**Figura 11 – Sequência da autenticação (pré-condição dos casos de uso)**




Fonte: elaborado pelo autor (2026).